

Project Code

✓ NLP Midterm Project - Group 02 - AI Content Detection

Dataset 1: DAIGT V2 Train Dataset

Dataset 1 Link: <https://www.kaggle.com/datasets/thedrcat/daigt-v2-train-dataset/data>

Dataset 2: HC3 (Human ChatGPT Comparison Corpus)

Dataset 2 Link: <https://huggingface.co/datasets/Hello-SimpleAI/HC3>

Algorithms: Multinomial Naive Bayes, Logistic Regression, LinearSVC

Representations: Bag of Words, TF-IDF

✓ Import libraries and download NLTK resources

```
import re
import pandas as pd
import numpy as np
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

✓ Extract Dataset from Google Drive Links

```
SAMPLE_PER_CLASS = 3000
path_daigt_download = 'https://drive.google.com/uc?export=download&id=1L1a51PNUSjmLA167XT3Yv-S55d9FaW0U'
path_hc3_download = 'https://drive.google.com/uc?export=download&id=1Dse8l3ct3elof5gqvVWcl0_wfm38RsV0'
print('Google Drive download links for DAIGT V2 and HC3 datasets are configured.')
```

✓ Read Dataset 1: DAIGT V2 Train Dataset

```
daigt_raw = pd.read_csv(path_daigt_download)

df1 = daigt_raw[['text', 'label']].dropna()
df1['label'] = df1['label'].astype(int)

print("=" * 60)
print("Dataset 1: DAIGT V2 Train Dataset")
print("Dataset Summary")
print("=" * 60)

print(f"Total Samples : {len(df1):,}")
print(f"Total Columns : {df1.shape[1]}")

print("\nClass Distribution")
print(df1['label']
      .value_counts()
      .sort_index()
      .rename_axis("Label")
      .reset_index(name="Number of Samples"))

print("\nFirst 5 Records")
```

```
print(df1.head())
```

✓ Read Dataset 2: HC3 (Human ChatGPT Comparison Corpus)

```
hc3_raw = pd.read_json(path_hc3_download, lines=True)

human = (
    hc3_raw[['human_answers']]
    .explode('human_answers')
    .rename(columns={'human_answers': 'text'})
)
human['label'] = 0

chatgpt = (
    hc3_raw[['chatgpt_answers']]
    .explode('chatgpt_answers')
    .rename(columns={'chatgpt_answers': 'text'})
)
chatgpt['label'] = 1

df2 = pd.concat([human, chatgpt], ignore_index=True).dropna()
df2['text'] = df2['text'].astype(str)

print("=" * 60)
print("Dataset 2: HC3 (Human ChatGPT Comparison Corpus)")
print("Dataset Summary")
print("=" * 60)

print(f"Total Samples : {len(df2):,}")
print(f"Total Columns : {df2.shape[1]}")

print("\nClass Distribution")
print(df2['label'].value_counts().sort_index().rename_axis("Label").reset_index(name="Number of Samples"))

print("\nFirst 5 Records")
print(df2.head())
```

✓ Balance both datasets by sampling equal rows per class

```
def balance(df, n, seed=42):
    parts = []
    for value in sorted(df['label'].unique()):
        subset = df[df['label'] == value]
        parts.append(subset.sample(n=min(n, len(subset)), random_state=seed))
    return pd.concat(parts).sample(frac=1, random_state=seed).reset_index(drop=True)

df1 = balance(df1, SAMPLE_PER_CLASS)
df2 = balance(df2, SAMPLE_PER_CLASS)

print("=" * 50)
print("Dataset 1: DAIGT V2 Train Dataset")
print("Class Distribution After Balancing")
print("=" * 50)
print(df1['label'].value_counts().sort_index().rename_axis("Label").reset_index(name="Number of Rows"))

print("\n" + "=" * 50)
print("Dataset 2: HC3 (Human ChatGPT Comparison Corpus)")
print("Class Distribution After Balancing")
print("=" * 50)
print(df2['label'].value_counts().sort_index().rename_axis("Label").reset_index(name="Number of Rows"))
```

✓ Preprocess text (case folding, punctuation removal, tokenization, stop word removal, lemmatization)

```
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = str(text).lower()
    text = re.sub(r'^a-z\s', ' ', text)
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(tok) for tok in tokens if tok not in stop_words and len(tok) > 1]
    return ' '.join(tokens)
```

```
df1['clean_text'] = df1['text'].apply(preprocess)
df2['clean_text'] = df2['text'].apply(preprocess)

print('Sample of Preprocessed Text for Dataset 1 (DAIGT V2):')
print(df1[['text', 'clean_text']].head())
print('\nSample of Preprocessed Text for Dataset 2 (HC3):')
print(df2[['text', 'clean_text']].head())
```

✓ Train / test split for both datasets

```
X1_train, X1_test, y1_train, y1_test = train_test_split(
    df1['clean_text'], df1['label'], test_size=0.2, random_state=42, stratify=df1['label'])

X2_train, X2_test, y2_train, y2_test = train_test_split(
    df2['clean_text'], df2['label'], test_size=0.2, random_state=42, stratify=df2['label'])

print(f'Dataset 1 (DAIGT V2) Training set size: {len(X1_train)} samples')
print(f'Dataset 1 (DAIGT V2) Testing set size: {len(X1_test)} samples')
print(f'Dataset 2 (HC3) Training set size: {len(X2_train)} samples')
print(f'Dataset 2 (HC3) Testing set size: {len(X2_test)} samples')
```

✓ Bag of Words (BoW) Feature Extraction

```
from sklearn.feature_extraction.text import CountVectorizer

bow1 = CountVectorizer()
X1_train_bow = bow1.fit_transform(X1_train)
X1_test_bow = bow1.transform(X1_test)

bow2 = CountVectorizer()
X2_train_bow = bow2.fit_transform(X2_train)
X2_test_bow = bow2.transform(X2_test)

print("=" * 70)
print("Bag of Words (BoW) Feature Extraction Summary")
print("=" * 70)

print("\nDataset 1: DAIGT V2 Train Dataset")
print(f"Training Feature Matrix Shape : {X1_train_bow.shape}")
print(f"Testing Feature Matrix Shape : {X1_test_bow.shape}")
print(f"Vocabulary Size : {len(bow1.vocabulary_):,}")

print("\nDataset 2: HC3 (Human ChatGPT Comparison Corpus)")
print(f"Training Feature Matrix Shape : {X2_train_bow.shape}")
print(f"Testing Feature Matrix Shape : {X2_test_bow.shape}")
print(f"Vocabulary Size : {len(bow2.vocabulary_):,}")
```

✓ TF-IDF Feature Extraction

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf1 = TfidfVectorizer()
X1_train_tfidf = tfidf1.fit_transform(X1_train)
X1_test_tfidf = tfidf1.transform(X1_test)

tfidf2 = TfidfVectorizer()
X2_train_tfidf = tfidf2.fit_transform(X2_train)
X2_test_tfidf = tfidf2.transform(X2_test)

print("=" * 70)
print("TF-IDF Feature Extraction Summary")
print("=" * 70)

print("\nDataset 1: DAIGT V2 Train Dataset")
print(f"Training Feature Matrix Shape : {X1_train_tfidf.shape}")
print(f"Testing Feature Matrix Shape : {X1_test_tfidf.shape}")
print(f"Vocabulary Size : {len(tfidf1.vocabulary_):,}")

print("\nDataset 2: HC3 (Human ChatGPT Comparison Corpus)")
print(f"Training Feature Matrix Shape : {X2_train_tfidf.shape}")
print(f"Testing Feature Matrix Shape : {X2_test_tfidf.shape}")
print(f"Vocabulary Size : {len(tfidf2.vocabulary_):,}")
```

✧ Evaluation helper and results container

```
results = {}
print("The 'results' dictionary will store evaluation metrics. The 'evaluate' function trains a model, makes predictions, and stores the results in the dictionary.")

def evaluate(model, X_train, y_train, X_test, y_test, title):
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    acc = accuracy_score(y_test, predictions)
    prec = precision_score(y_test, predictions, average='weighted', zero_division=0)
    rec = recall_score(y_test, predictions, average='weighted', zero_division=0)
    f1 = f1_score(y_test, predictions, average='weighted', zero_division=0)
    print(title)
    print(classification_report(y_test, predictions))
    print('Accuracy:', acc)
    print('-' * 60)
    return {'Accuracy': acc, 'Precision': prec, 'Recall': rec, 'F1': f1, 'y_true': y_test, 'y_pred': predictions}

print("'evaluate' function defined and ready for use.")
```

✧ Naive Bayes (MultinomialNB)

```
results[('Naive Bayes', 'BoW', 'Dataset 1')] = evaluate(MultinomialNB(), X1_train_bow, y1_train, X1_test_bow, y1_test, 'Naive Bayes (BoW) - Dataset 1')
results[('Naive Bayes', 'TF-IDF', 'Dataset 1')] = evaluate(MultinomialNB(), X1_train_tfidf, y1_train, X1_test_tfidf, y1_test, 'Naive Bayes (TF-IDF) - Dataset 1')
results[('Naive Bayes', 'BoW', 'Dataset 2')] = evaluate(MultinomialNB(), X2_train_bow, y2_train, X2_test_bow, y2_test, 'Naive Bayes (BoW) - Dataset 2')
results[('Naive Bayes', 'TF-IDF', 'Dataset 2')] = evaluate(MultinomialNB(), X2_train_tfidf, y2_train, X2_test_tfidf, y2_test, 'Naive Bayes (TF-IDF) - Dataset 2')
print('Naive Bayes model evaluation results stored in the results dictionary.')
```

✧ Confusion matrices for Naive Bayes

```
combos = [('BoW', 'Dataset 1'), ('TF-IDF', 'Dataset 1'), ('BoW', 'Dataset 2'), ('TF-IDF', 'Dataset 2')]
for rep, ds in combos:
    scores = results[('Naive Bayes', rep, ds)]
    ConfusionMatrixDisplay(confusion_matrix(scores['y_true'], scores['y_pred']), display_labels=['human', 'ai']).plot()
    plt.title('Naive Bayes' + ' - ' + rep + ' - ' + ds)
    plt.show()
print('Confusion matrices for Naive Bayes models displayed above.')
```

✧ Logistic Regression

```
results[('Logistic Regression', 'BoW', 'Dataset 1')] = evaluate(LogisticRegression(max_iter=1000), X1_train_bow, y1_train, X1_test_bow, y1_test, 'Logistic Regression (BoW) - Dataset 1')
results[('Logistic Regression', 'TF-IDF', 'Dataset 1')] = evaluate(LogisticRegression(max_iter=1000), X1_train_tfidf, y1_train, X1_test_tfidf, y1_test, 'Logistic Regression (TF-IDF) - Dataset 1')
results[('Logistic Regression', 'BoW', 'Dataset 2')] = evaluate(LogisticRegression(max_iter=1000), X2_train_bow, y2_train, X2_test_bow, y2_test, 'Logistic Regression (BoW) - Dataset 2')
results[('Logistic Regression', 'TF-IDF', 'Dataset 2')] = evaluate(LogisticRegression(max_iter=1000), X2_train_tfidf, y2_train, X2_test_tfidf, y2_test, 'Logistic Regression (TF-IDF) - Dataset 2')
print('Logistic Regression model evaluation results stored in the results dictionary.')
```

✧ Confusion matrices for Logistic Regression

```
combos = [('BoW', 'Dataset 1'), ('TF-IDF', 'Dataset 1'), ('BoW', 'Dataset 2'), ('TF-IDF', 'Dataset 2')]
for rep, ds in combos:
    scores = results[('Logistic Regression', rep, ds)]
    ConfusionMatrixDisplay(confusion_matrix(scores['y_true'], scores['y_pred']), display_labels=['human', 'ai']).plot()
    plt.title('Logistic Regression' + ' - ' + rep + ' - ' + ds)
    plt.show()
print('Confusion matrices for Logistic Regression models displayed above.')
```

✧ Support Vector Machine (LinearSVC)

```
results[('Support Vector Machine', 'BoW', 'Dataset 1')] = evaluate(LinearSVC(max_iter=5000), X1_train_bow, y1_train, X1_test_bow, y1_test, 'Support Vector Machine (BoW) - Dataset 1')
results[('Support Vector Machine', 'TF-IDF', 'Dataset 1')] = evaluate(LinearSVC(max_iter=5000), X1_train_tfidf, y1_train, X1_test_tfidf, y1_test, 'Support Vector Machine (TF-IDF) - Dataset 1')
results[('Support Vector Machine', 'BoW', 'Dataset 2')] = evaluate(LinearSVC(max_iter=5000), X2_train_bow, y2_train, X2_test_bow, y2_test, 'Support Vector Machine (BoW) - Dataset 2')
results[('Support Vector Machine', 'TF-IDF', 'Dataset 2')] = evaluate(LinearSVC(max_iter=5000), X2_train_tfidf, y2_train, X2_test_tfidf, y2_test, 'Support Vector Machine (TF-IDF) - Dataset 2')
print('Support Vector Machine model evaluation results stored in the results dictionary.')
```

✧ Confusion matrices for Support Vector Machine

```
combos = [('BoW', 'Dataset 1'), ('TF-IDF', 'Dataset 1'), ('BoW', 'Dataset 2'), ('TF-IDF', 'Dataset 2')]
for rep, ds in combos:
```

```

scores = results[('Support Vector Machine', rep, ds)]
ConfusionMatrixDisplay(confusion_matrix(scores['y_true'], scores['y_pred']), display_labels=['human', 'ai']).plot()
plt.title('Support Vector Machine' + ' - ' + rep + ' - ' + ds)
plt.show()
print('Confusion matrices for Support Vector Machine models displayed above.')

```

▼ Detailed metrics per algorithm (Accuracy, Precision, Recall, F1)

```

combos = [('BoW', 'Dataset 1'), ('TF-IDF', 'Dataset 1'), ('BoW', 'Dataset 2'), ('TF-IDF', 'Dataset 2')]

for algo in ['Naive Bayes', 'Logistic Regression', 'Support Vector Machine']:
    table_rows = []
    for rep, ds in combos:
        scores = results[(algo, rep, ds)]
        table_rows.append({
            'Representation': rep,
            'Dataset': ds,
            'Accuracy': round(scores['Accuracy'], 4),
            'Precision': round(scores['Precision'], 4),
            'Recall': round(scores['Recall'], 4),
            'F1': round(scores['F1'], 4)
        })
    print(algo)
    print(pd.DataFrame(table_rows).to_string(index=False))
    print('=' * 70)

```

▼ Final result table (accuracy)

```

rows = []
for algo in ['Naive Bayes', 'Logistic Regression', 'Support Vector Machine']:
    for rep in ['BoW', 'TF-IDF']:
        rows.append({
            'Algorithm': algo,
            'Representation': rep,
            'Dataset 1': round(results[(algo, rep, 'Dataset 1')]['Accuracy'], 4),
            'Dataset 2': round(results[(algo, rep, 'Dataset 2')]['Accuracy'], 4)
        })

final_table = pd.DataFrame(rows)
print(final_table.to_string(index=False))

```

▼ Export sample dataset file 1 (DAIGT V2) - first 100 rows, all 5 columns

```

sample1 = daigt_raw.head(100)

print("=" * 60)
print("Sample Dataset 1 (DAIGT V2) for Submission")
print("=" * 60)
print(f"Rows      : {sample1.shape[0]}")
print(f"Columns   : {sample1.shape[1]} -> {list(sample1.columns)}")
print()
print(sample1.to_string(max_colwidth=30))

```

▼ Export sample dataset file 2 (HC3) - first 100 rows, all 5 columns

```

sample2 = hc3_raw.head(100)

print("=" * 60)
print("Sample Dataset 2 (HC3) for Submission")
print("=" * 60)
print(f"Rows      : {sample2.shape[0]}")
print(f"Columns   : {sample2.shape[1]} -> {list(sample2.columns)}")
print()
print(sample2.to_string(max_colwidth=30))

```

